

Documentación Técnica: Servicio DNS Distribuido mediante Java RMI

1. Diseño de la Interfaz Remota

La interfaz `IDNSService` se ha diseñado siguiendo los principios de abstracción y expresividad exigidos en sistemas distribuidos:

- **Abstracción de Red:** El cliente interactúa con métodos de alto nivel como `query()` y `add()`, abstrayéndose por completo de la lógica interna del servidor (uso de mapas o gestión de memoria).
- **Herencia de Remote:** Se utiliza para indicar a la JVM que los métodos pueden ser invocados desde máquinas distintas a la que reside el objeto, permitiendo la conexión distribuida.
- **Gestión de RemoteException:** A diferencia de los métodos locales, cada operación obliga al programador a gestionar explícitamente fallos de red, latencia o timeouts mediante bloques *try-catch*, garantizando la robustez del sistema.
- **Tipado Fuerte y Serialización:** Al usar tipos nativos (`String`, `int`), Java RMI se encarga de la serialización (paso de parámetros) de forma transparente, eliminando errores de formato comunes en implementaciones manuales con Sockets.

2. Decisiones Adoptadas

Para cumplir con los requisitos de funcionalidad y concurrencia, se han tomado las siguientes decisiones de diseño:

- **Estado Compartido (Singleton):** Siguiendo el anexo técnico, se ha expuesto una única instancia del objeto remoto para que todos los clientes compartan la misma caché de registros DNS, facilitando la consistencia de los datos.
- **Uso de ConcurrentHashMap:** Dado que RMI es multihilo por naturaleza, se ha implementado esta estructura para permitir que múltiples clientes lean y escriban de forma simultánea y segura, evitando condiciones de carrera sin bloquear el sistema.
- **Clase Registro y Gestión de TTL:** Se ha centralizado la información (IP y tiempo de expiración) en un objeto interno. El servidor utiliza una **gestión pasiva del TTL**, comprobando la validez del registro solo cuando se solicita, lo que optimiza el uso de CPU.
- **Identidad de Red y Registro:** Se utiliza `System.setProperty("java.rmi.server.hostname", ...)` para garantizar que el stub enviado al cliente contenga la IP correcta para comunicaciones fuera de *localhost*.
- **Publicación con rebind:** Se emplea `Naming.rebind` para que, en caso de reinicio del servidor, el registro en el RMI Registry (puerto 1099) se actualice automáticamente sin lanzar excepciones por nombre duplicado.

3. Reflexión Técnica: Cambio de Paradigma

A. Del Protocolo a los Métodos

- **Modelo previo (Sockets):** La comunicación era manual y basada en flujos de texto. Era necesario diseñar tramas (ej. "ADD:dominio:ip") y parsearlas mediante `split()`.
- **Modelo actual (RMI):** La red se vuelve invisible. El programador se centra en la lógica de negocio invocando métodos directamente, mientras RMI gestiona el transporte.

B. Problemas Resueltos

- **Marshalling Automático:** Java empaqueta y envía los objetos automáticamente; ya no es necesario convertir datos a bytes o strings de forma manual.
- **Multihilo Transparente:** RMI gestiona su propio *pool* de hilos para atender a varios clientes, eliminando la necesidad de crear Threads manualmente en el servidor.

C. Nuevos Desafíos

- **Latencia y Bloqueo:** Al ser llamadas remotas, el programa cliente se detiene esperando la respuesta de la red, un factor que no existe en llamadas locales y debe ser considerado en el diseño de la UI.
- **Robustez Obligatoria:** La arquitectura distribuida introduce puntos de fallo externos (caídas de red, cortafuegos), lo que hace obligatorio un manejo de excepciones mucho más estricto.

4. Instrucciones de Ejecución

Paso 1: Compilación

Desde la terminal en la raíz del proyecto: `javac *.java`

Paso 2: Lanzamiento del Servidor

Es fundamental indicar la IP pública/privada de la interfaz de red que escuchará las peticiones: `java -Djava.rmi.server.hostname=192.168.X.X ServidorDNS`

Paso 3: Lanzamiento del Cliente

Ejecutar el cliente pasando la IP del servidor como argumento: `java ClienteDNS 192.168.X.X`